

2026년 산업맞춤 단기직무능력인증과정

클라우드와 AI 패러다임 전환

1주차 · 강의 및 실습 자료

메타넷디지털 서경진

주관 교육부 Ministry of Education | 국가평생교육진흥원

주차 개요 — 3개 주제

이번 주차에서 다룰 3개 핵심 주제를 한눈에 살펴봅니다.

1 클라우드 + AI 전환

- 클라우드/AI 결합 흐름
- IaaS·PaaS·SaaS
- AI Native 개념

2 AI 생태계 소개

- 모델·프레임워크·플랫폼
- 주요 벤더와 오픈소스
- LLM 생태계 지형

3 AI 개발 환경 구성

- Python/IDE/노트북
- API 키·가상환경
- Azure 개발 환경

01

1주차 | SECTION

클라우드 + AI 패러다임 전환

온프레미스에서 클라우드 네이티브 AI로 — 왜 지금 전환인가.

학습 목표

클라우드와 AI가 결합되는 거시적 흐름과 그 의미를 살펴봅니다.

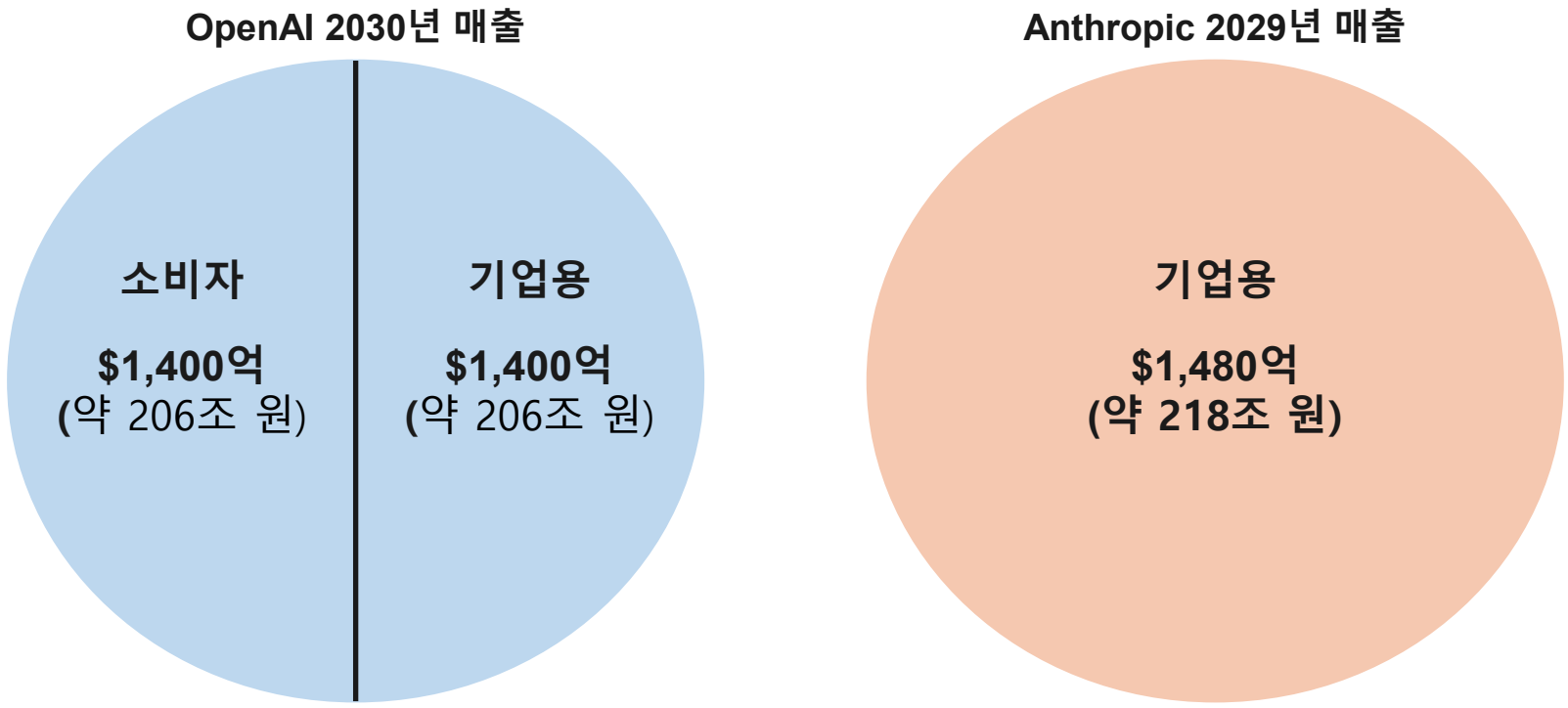
- 클라우드 컴퓨팅의 핵심 개념과 서비스 모델(IaaS/PaaS/SaaS)을 설명한다
  - 클라우드 5대 특성(NIST): 온디맨드 셀프서비스·광대역 접속·자원 풀링·신속한 탄력성·측정 가능 서비스
- AI 워크로드가 클라우드로 이동하는 기술·비용적 배경을 이해한다
- AI Native 아키텍처의 특징을 사례 중심으로 설명한다

출처 *NIST SP 800-145 (클라우드 컴퓨팅 정의) · Google Cloud "Introduction to Generative AI"(YouTube)*

Market Trends

AI 시장 리더인 OpenAI와 Anthropic의 AI Agent 시장의 매출 전망입니다.

OpenAI와 Anthropic AI Agent 영역 두 거인의 야심찬 매출 전망



출처 OpenAI·Anthropic AI 에이전트 매출 전망(업계 보도·시장 분석 종합)

Market Trends

Palantir가 대중화시킨 FDE(현장 배포 엔지니어)는 고객이 AI Agent 프로젝트에서 가치를 얻도록 보장합니다.

FDE (Forward Deployed Engineer) 정의 및 장단점

- **FDE의 역할 = 소프트웨어 엔지니어 + 솔루션 아키텍트 + 고객에 내재된 제품 빌더**

회사	FDE 동급 역할
Anthropic	Forward Deployed Engineer
OpenAI	Forward Deployed Engineer
Sierra	Forward Deployed Agent Engineer
Glean	AI Outcomes Package

장점 (Pros)

빠른 가치 실현 시간
강력한 고객 관계 구축

단점 (Cons)

사업 확장의 어려움
높은 운영 비용

출처 Palantir Forward Deployed Engineer(2005 창시) · The New Stack "Why OpenAI and Anthropic are hiring FDE"(2025)

Market Trends

비즈니스를 위한 가격 모델이 “좌석 기반”에서 “성과 기반”으로 전환됩니다.

비즈니스 가격 모델의 전환

- **스타트업이 기존 업체에 도전하는 데 활용 가능한 역포지셔닝(Counter-positioning) 전략.**
- **기존 업체는 현재 사업을 잠식하게 되므로 비즈니스 모델 전환이 어려울 것임.**

항목	좌석 기반 요금제	성과 기반 요금제
결제 기준	사용자 수	완료된 작업 또는 결과
예시	사용자당 월 $100	작업·성과당 $5
가치 연계	약함 (사용자 수 기준 결제)	강함 (결과 기준 결제)

출처 SaaS 좌석 기반 → 성과 기반(Outcome-based) 과금 전환(업계 동향)

Market Trends

AI 에이전트 오케스트레이션의 부상

멀티 에이전트 오케스트레이션의 개념



무엇을 하는가?

- 계획 및 분해: 어떤 에이전트가 무엇을 할지 결정
- 에이전트 선택: 작업에 적합한 에이전트 선택
- 상태 및 메모리 관리: 진행 상황 추적
- 종합: 결과 생성

LLM은 범용재임. 진정한 가치는 인간의 개입 없이 4시간짜리 작업을 안정적으로 수행하는 오케스트레이션 레이어에 있음

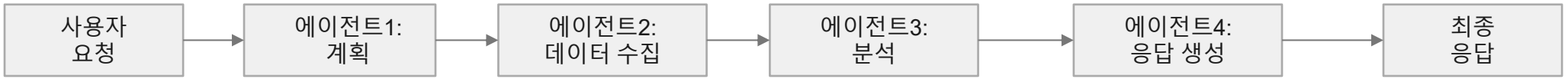
출처 Anthropic "Building Effective Agents"(2024) · AI 에이전트 오케스트레이션 동향

Market Trends

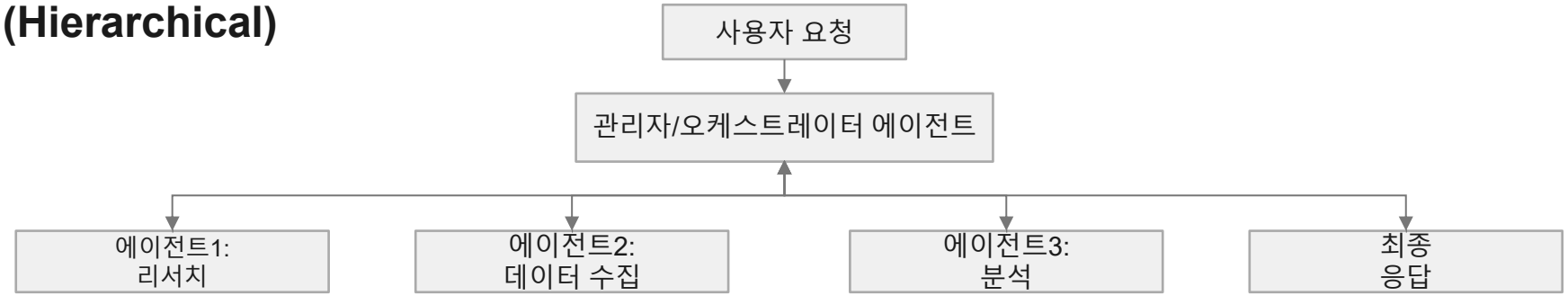
AI 에이전트 오케스트레이션 패턴

Orchestration Patterns

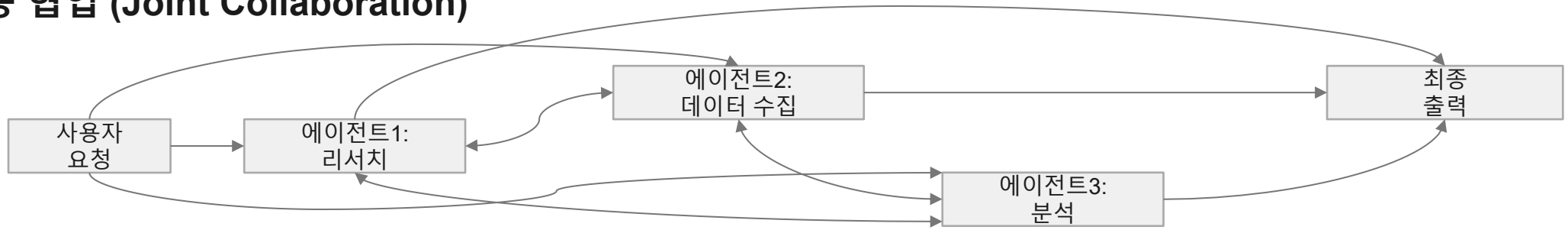
순차적 (Sequential)



계층적 (Hierarchical)



공동 협업 (Joint Collaboration)



출처 *Anthropic "Building Effective Agents"(2024) — 오케스트레이션·워크플로 패턴*

Tech Trends

지능형 위임(Intelligent Delegation): Google DeepMind 연구 논문

지능형 위임(Intelligent Delegation)

지능형 위임(Intelligent Delegation)은 AI 에이전트가 사용자를 대신하여 자율적으로 행동하기 위해 충분한 정보, 권한, '가치 정렬'을 갖추었는지 판단하는 프레임워크임

항목	전통적 자동화	지능형 위임
입력	명시적 지시	고수준 목표
오류 처리	충돌 또는 '잘못된' 규칙 수행	일시 중지 후 '의도' 확인
의사결정 로직	결정론적 (If/Then)	확률론적 (위험/보상)
사용자 상호작용	단방향 명령	양방향 조율

장점 (Pros)

감사 가능성 (Auditability)

단점 (Cons)

처리 속도 저하 (Slowness)

출처 *Tomašev, Franklin & Osindero, "Intelligent AI Delegation"(Google DeepMind, arXiv:2602.11865, 2026)*

Tech Trends - Landscape

Vertical AI Agents, Horizontal AI Agents, AI Agent Infrastructure 별로 분류합니다.

Vertical AI Agents

제조 (Manufacturing)	금융 (Financial Services)
물류 (Supply Chain & Logistics)	리테일 (Retail)

Horizontal AI Agents

Customer Support	Intelligent Document Processing
Knowledge & Search	Data Analytics

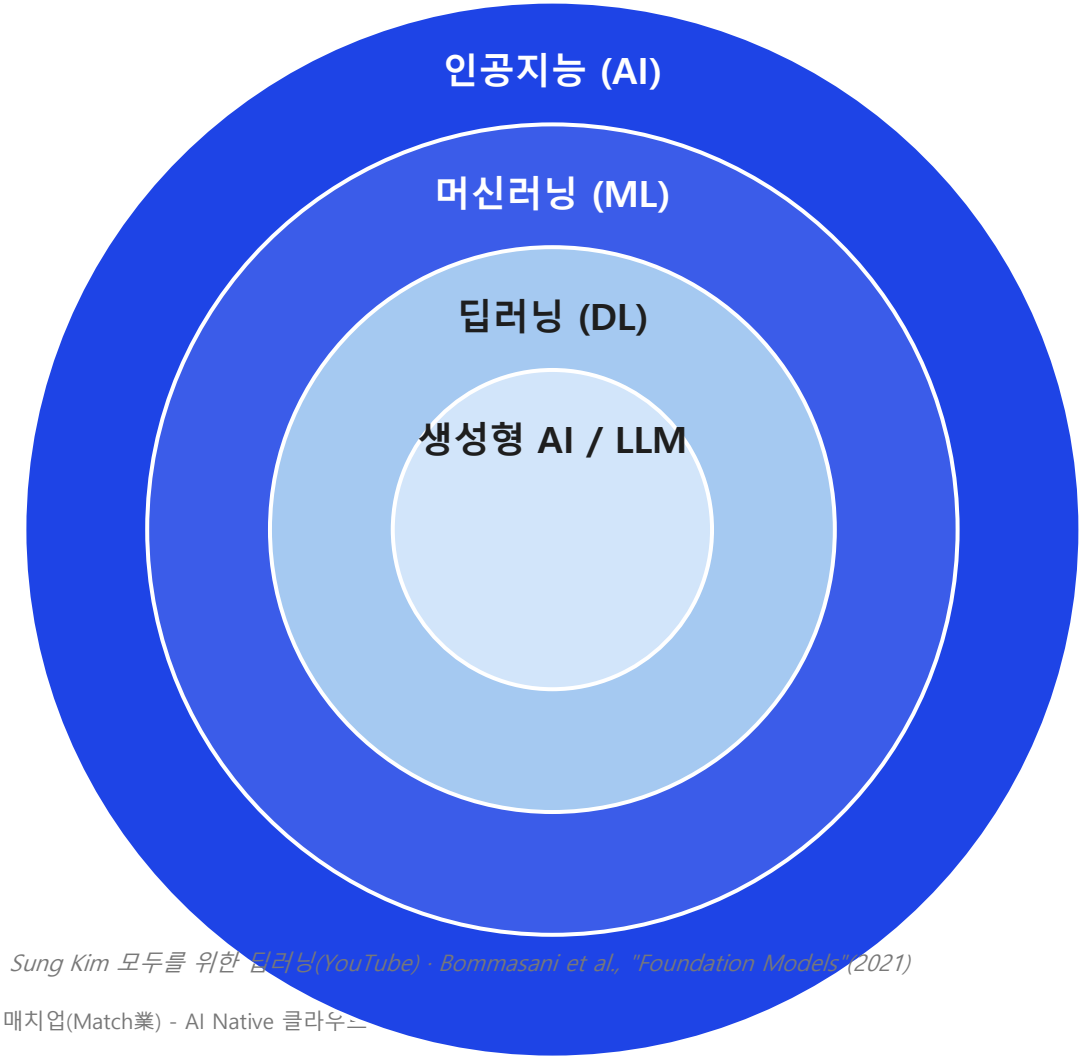
AI Agent Infrastructure

Build	Orchestration
Governance & Guardrails	Security

출처 CB Insights "AI Agent Market Map"(2025) · Insight Partners "State of the AI Agent Ecosystem"

AI · ML · DL · 생성형 AI 관계

가장 넓은 개념이 AI이고, 그 안에 ML → DL → 생성형 AI가 포함됩니다.



인공지능 (AI)
인간의 지능을 모사하는 모든 기술

머신러닝 (ML)
데이터로 규칙을 스스로 학습

딥러닝 (DL)
신경망 기반 표현 학습

생성형 AI / LLM
콘텐츠를 생성하는 모델

출처 Sung Kim 모두를 위한 딥러닝(YouTube) · Bommasani et al., "Foundation Models"(2021)

클라우드 책임 분담 모델

위로 갈수록 제공자 관리 영역이 커지고, 사용자 운영 부담은 줄어듭니다.

SaaS 애플리케이션까지 제공자가 관리 (예: M365, Copilot)

PaaS 런타임·미들웨어 제공, 앱 코드에 집중 (예: App Service)

IaaS OS 위쪽은 사용자가 관리 (예: VM)

On-Premise 모든 계층을 사용자가 직접 관리

핵심 설명

1. **SaaS**
   완성된 SW를 구독해 사용 — 운영 부담 최소(예: M365)
2. **PaaS**
   런타임·미들웨어 제공 — 앱 코드 개발에 집중(예: App Service)
3. **IaaS**
   VM·스토리지만 빌려 OS 위 계층은 직접 관리
4. **선택 기준**
   통제 범위와 운영 부담의 트레이드오프로 결정
5. **AI 워크로드**
   대부분 PaaS/SaaS(AIaaS)로 빠르게 시작

출처 NIST SP 800-145 (클라우드 컴퓨팅 정의)

클라우드 서비스 모델

클라우드 서비스는 관리 범위에 따라 IaaS·PaaS·SaaS·AIaaS로 구분됩니다.

1 IaaS

- 가상머신·스토리지·네트워크
- OS/런타임은 사용자 책임
- 예: Azure VM, AWS EC2

2 PaaS

- 플랫폼 제공, 인프라 추상화
- 앱 코드에 집중
- 예: App Service, AKS

3 SaaS

- 완성된 SW 구독
- 운영 부담 최소화
- 예: M365, Copilot

4 AI as a Service

- 모델을 API로 소비
- RAG·파인튜닝으로 확장
- 예: Azure OpenAI

출처 *NIST SP 800-145 (클라우드 컴퓨팅 정의)*

전통 IT vs X-AI Native

개발 방식이 사람 중심 코딩에서 AI 협업·명세 주도(X+AI)로 전환됩니다.

전통적 IT

- 사람이 코드를 직접 작성
- 규칙 기반 로직
  - 폭포수·긴 개발 주기
- 문서는 사후 작성

X-AI Native

- AI 에이전트와 협업 개발
- 데이터·모델 기반 구현
  - 명세 주도(SDD)·짧은 반복
- 검증 자동화

출처 2026 매치업 X+AI 교육과정

전통 IT vs X-AI Native

고정 인프라·수동 운영에서 탄력적 클라우드·자동화 운영으로 전환됩니다.

전통적 IT

- 고정 인프라·수동 스케일
- 온프레미스 중심
  - 수동 배포·긴 리드타임
- 인프라 운영 부담

X-AI Native

- 탄력적 클라우드 자원
- 클라우드 네이티브
  - CI/CD 자동 배포
- 관측성·자동화 운영

출처 2026 매치업 X+AI 교육과정 · NIST SP 800-145

전통 IT vs X-AI Native

X-AI Native는 AI 도구로 생산성·일관성·실험 속도를 동시에 끌어올립니다.

전통적 IT

- 개인 역량에 의존
- 반복 작업 수작업
  - 느린 실험·검증
- 비용·시간 부담 큼

X-AI Native

- 도구·컨텍스트로 표준화
- 반복 작업 자동화
  - 빠른 실험·자동 검증
- 생산성·품질 향상

출처 2026 매치업 X+AI 교육과정 · AI 코딩 솔루션 비교분석(사내)

전통 IT vs X-AI Native

Monolith 구조의 전통 IT 시스템의 인프라/아키텍처를 MSA 구조의 X-AI Native Application으로 현대화 전환하고 MCP 기능을 추가하여 AI 에코시스템과 연동이 용이하도록 설계 및 개발합니다.

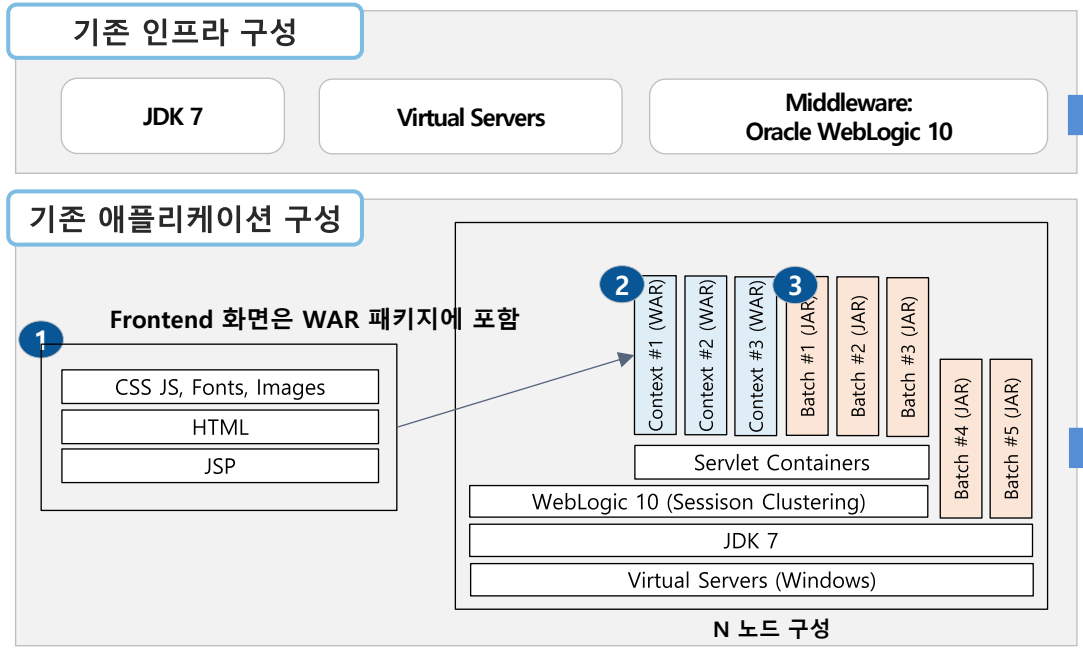
	Monolith 구조의 App 구성 (AS-IS)		MSA 구조의 Cloud Native Application + MCP 구성 (TO-BE)		
WEB	사내 영역(INT)	별도 웹 서버를 사용하여 구성하지 않음	사내 영역(INT)	• Vue3 기반 MFA Frontend 개발 및 구성 • Container 기반 Frontend Nginx 웹 서버 구성	Kubernetes Hybrid Cloud
WAS	▪ Frontend와 Backend 분리 없이 Monolith 방식으로 개발됨 ▪ VM 서버에 미들웨어를 구성하고 Monolith 구조의 App 배포		▪ Frontend와 Backend 분리한 MSA 방식으로 개발됨 ▪ Containerization을 통해 마이크로 서비스로 K8S에 App 배포		
	Java 런타임	Java Development Kit 7	Java 런타임	Java Development Kit 17 이상	
	미들웨어	Oracle WebLogic 10	미들웨어	Embedded Tomcat, Webflux	
	개발언어	JSP + Spring + Mybatis (SDP)	개발언어	Vue.js + **Spring Boot/Cloud/AI** + JPA/Mybatis	
AI	▪ **N/A**		▪ **AI Agent/Workflow/LLM/MCP 등 AI 서비스를 위한 에코시스템 구성 및 연동** ▪ **Flowise + 다중 AI Agent 오케스트레이션 + AI Agent + MCP 연동**		
세션 및 캐시	▪ 미들웨어에서 제공하는 Session Clustering 기능 사용 가능 ▪ 분산 환경을 위한 캐시는 사용하지 않음		▪ MSA 분산 환경 세션 관리를 위해 VM 기반 Redis Cluster 구성 ▪ Redis Cluster를 MSA 마이크로 서비스 분산 캐시로 사용		Hybrid Cloud
연동	▪ 대상 시스템에 클라이언트 단위로 제어되지 않는 요청 부하 발생		▪ Spring Cloud에서 제공하는 클라이언트 로드발란서 기능을 활용하여 제어가 가능한 요청 부하 발생		Hybrid Cloud
배치	▪ VM 환경 WebLogic 미들웨어에서 구동하는 온라인 배치 활용 ▪ VM에 Crontab으로 등록된 Java 배치 프로그램 활용		▪ K8S의 Container 기반 Cronjob/Job을 활용한 배치 구성 ▪ Spring Batch 기반 Container로 On-demand 배치 프로그램 구성		Kubernetes Hybrid Cloud
DB	▪ Oracle 11g 기반 Procedure, Function, SQL 쿼리 활용		▪ Cloud VM 기반 종량제 Oracle 19c RAC 클러스터 구성 ▪ Oracle 19c 기반 Procedure, Function, SQL 쿼리로 전환 (성능 향상)		Hybrid Cloud

출처 2026 매치업 X+AI 교육과정

전통 IT vs X-AI Native

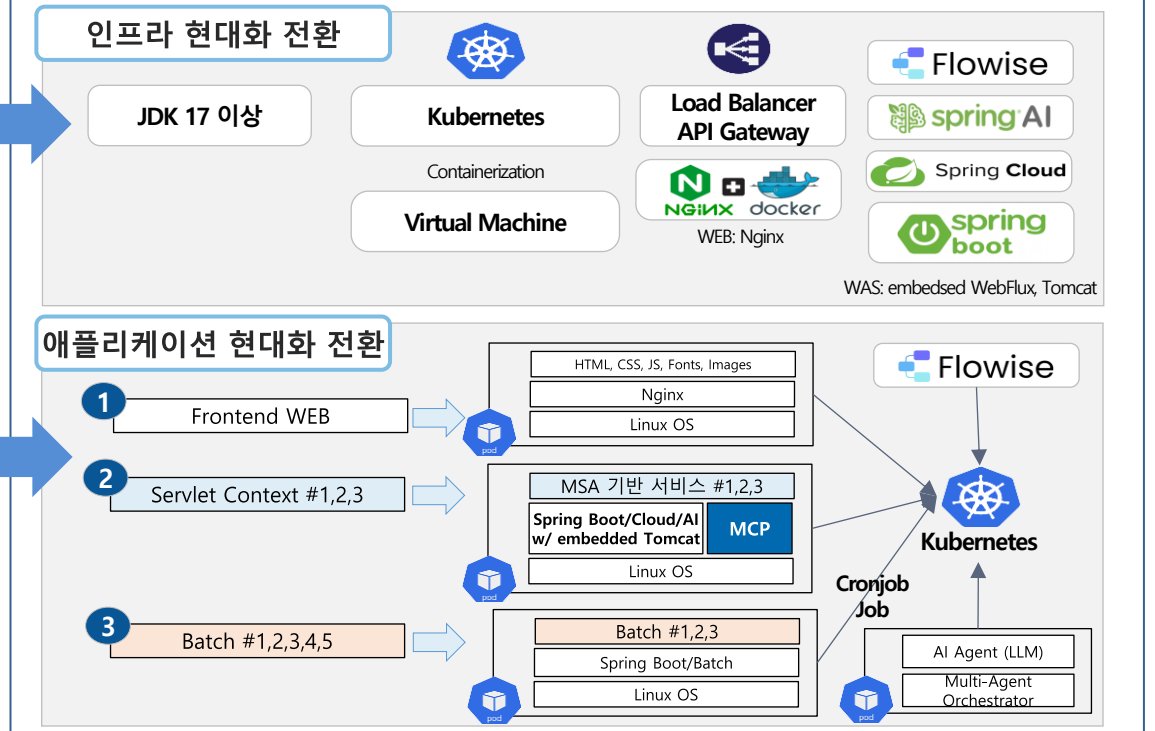
전통 IT 시스템의 인프라/애플리케이션 현대화는 X-AI 최신 기술을 적용하여 전환을 진행합니다.

Monolith 구조의 App 구성 (AS-IS)



- Virtual Server를 기반으로 Monolith 구조의 WAS 환경 구성
- WAS에 다수의 WAR와 JAR를 배포하여 각 모듈별 배포가 전체 서비스에 영향을 미침
- 서비스 성능 향상을 위해 VM Scale 확장 시 상당한 시간이 소요됨
- DevOps 기반 자동 빌드 및 배포하는데 상당히 긴 시간이 소요됨
- 빈번한 빌드 및 배포와 적은 코드 수정을 위한 체계가 미흡함

MSA 구조의 Cloud Native Application 구성 (TO-BE)



- Containerization을 통해 Kubernetes와 Serverless 환경으로 전환
- 전통 IT 시스템의 서비스에 영향 없이 MSA로 분리된 각 서비스 단위로 빌드 및 배포 수행
- 신속하게 Scale-in/out 가능, DevOps 파이프라인을 통해 신속하게 각 서비스별 빌드 및 배포 가능
- 빈번한 빌드 및 배포와 적은 코드 수정을 반영하기 용이함
- AI Agent/Workflow/LLM/MCP 등 AI 연동이 용이하도록 Kubernetes 기반 AI 에코시스템 구축

출처 *2026 매치업 X+AI 교육과정*

AI 코딩 도구 연동 실습 개요

Claude Desktop·Claude Code에 GPT-OSS·MCP를 연동하는 실습을 프롬프트·명령으로 진행합니다.

1 Claude Desktop

- 데스크톱 앱
- MCP 호스트
- 설정 파일로 연동

2 Claude Code

- 터미널 Agentic CLI
- 프롬프트로 명령 실행
- 9p 콘솔 실습 자동화

3 GPT-OSS

- 오픈 가중치 모델
- Ollama로 로컬 실행
- OpenAI 호환 API

4 MCP

- 도구·데이터 표준 연결
- config에 서버 등록
- Claude가 도구 사용

출처 Anthropic Claude Docs · MCP(2024) · OpenAI gpt-oss · Ollama 문서

1주차 실습 구성 — Claude Code 프롬프트 기반

13개 HANDS-ON을 6개 통합 실습으로 재구성 · 상세: prompt/1주차_실습_프롬프트.md

구성 로컬 Ollama·MCP·Claude Code로 수행 · 산출물: 실습산출물/1주차/

통합 실습 목록

- 1-1 Desktop·MCP 설정 + 인기 MCP 3종 등록
- 1-2 Azure AI 서비스 콘솔 조회 자동화
- 1-3 GPT-OSS 로컬 실행 + MCP 통합
- 1-4 로컬 Ollama 다중 모델 비교·라우팅
- 1-5 개발환경→첫 호출→견고화
- 1-6 1주차 실습 요약 문서 생성



실습 1-1 · Claude Desktop MCP 설정 + 3종 등록

설정 파일 생성 + Context7·Sequential Thinking·Playwright 멱등 등록

실습 목표 MCP 설정 파일 생성 후 인기 MCP 3종을 한 번에 멱등 등록

진행 단계

1. config 경로 확인·없으면 생성
2. 3종 MCP를 '없는 것만' 병합 등록
3. Desktop 재시작·권한 승인
4. 각 MCP 구동 확인

프롬프트

```
config 경로 확인, 없으면 mcpServers 생성 후
아래 3종을 '없는 것만' 병합(있으면 건너뜀):
 (1) context7
 (2) sequential-thinking
 (3) playwright
결과 표 + 실습산출물/1주차/1-1_mcp_setup.md 저장
```

02

1주차 | SECTION

AI 생태계 소개

모델, 프레임워크, 플랫폼 — AI 생태계의 지형도를 그린다.

AI 생태계 4계층

AI 생태계는 인프라–모델–프레임워크–애플리케이션의 4계층으로 이해합니다.

1 인프라/하드웨어

- GPU·TPU, 클라우드
- NVIDIA, 하이퍼스케일러

2 모델(Foundation)

- LLM·멀티모달 모델
- GPT, Claude, Llama

3 프레임워크/도구

- LangChain, LlamaIndex
- HuggingFace, 벡터DB

4 애플리케이션

- 챗봇·코파일럿·에이전트
- AI 코딩 도구(Claude Code 등)
- 업무 자동화 서비스

출처 *Bommasani et al., "On the Opportunities and Risks of Foundation Models"(2021) · Sung Kim 모두를 위한 딥러닝(YouTube)*

AI 패러다임의 발전

규칙 → 학습 → 딥러닝 → LLM → 에이전트로 진화해 왔습니다.



핵심 설명

1. **규칙 기반**
   사람이 if-then 규칙을 직접 작성
2. **통계 ML**
   데이터에서 패턴을 학습
3. **딥러닝**
   신경망으로 표현을 자동 학습
4. **LLM**
   대규모 언어모델이 범용 능력 확보
5. **에이전틱**
   LLM이 도구를 쓰고 스스로 계획·실행

출처 Bommasani et al.(2021) · Sung Kim 딥러닝 강의(YouTube)

AI 생태계 4계층 아키텍처

하위 인프라 위에 모델 → 프레임워크 → 애플리케이션이 쌓입니다.



핵심 설명

1. **애플리케이션**
   챗봇·코파일럿·에이전트 등 최종 서비스 계층
2. **프레임워크**
   LangChain·VectorDB 등 개발 도구 계층
3. **모델**
   GPT·Claude 등 파운데이션 모델
4. **인프라**
   GPU·클라우드 등 연산 기반
5. **계층 관계**
   하위가 상위를 떠받치는 스택 구조

출처 CNCF Cloud Native AI 백서 · Bommasani et al.(2021)

주요 파운데이션 모델

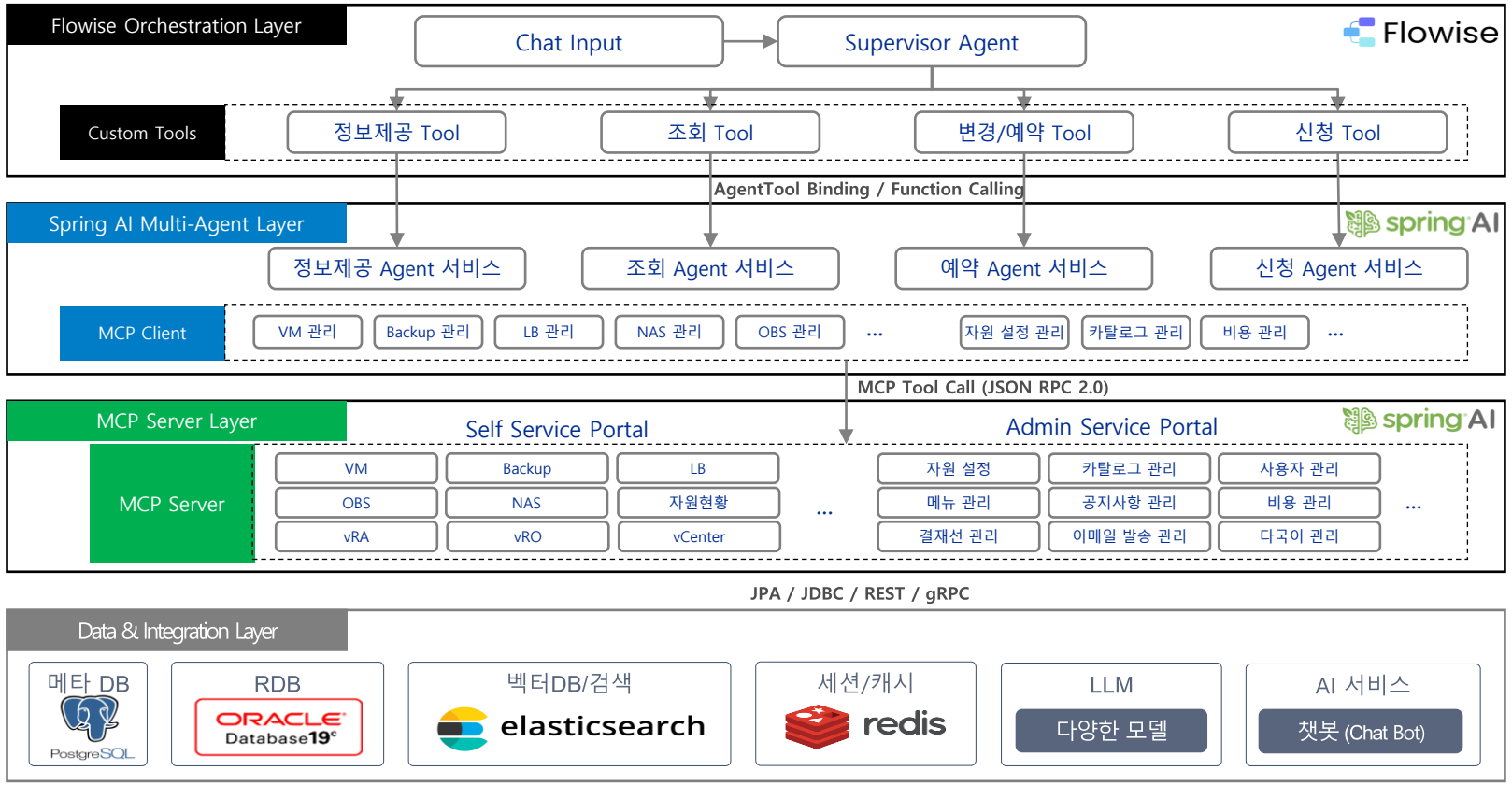
용도·강점·라이선스에 따라 적합한 파운데이션 모델을 선택합니다.

제공자	대표 모델	특징
OpenAI / Azure	GPT-4o	범용 멀티모달, 생태계 풍부
Anthropic	Claude (Opus/Sonnet)	장문·추론·안전성 강점
Google	Gemini	멀티모달·검색 연계
Meta	Llama 3	오픈 가중치, 온프레미스 가능

AI 에코시스템 구성 방안

기존 시스템에 다양한 AI 기능과 서비스를 제공하기 위해 Flowise + Spring AI + MCP를 기반으로 AI 에코시스템을 구성하여 Java와 No/Low Code 기반 개발 및 연동 환경을 유연하게 제공합니다.

AI 에코시스템 아키텍처 구성도



구성 방안

Workflow/Orchestration 적용

- ✓ AI Workflow 기반 업무 자동화:
  기존 서비스 포탈 업무를 AI Agent 기반으로 자동화하기 위해 Workflow를 쉽게 구성/배포하는 오픈 소스 솔루션 적용
- ✓ AI Orchestrator 기반 다중 Agent 연동:
  Spring AI를 기반으로 개발된 다중 업무 AI Agent 서비스를 Orchestration하고 연동하는 기능을 제공함

Spring AI 기반 MCP 적용

- ✓ MCP를 통한 Agentic AI 구현:
  Java와 Spring에 익숙한 개발자가 AI Agent, MCP Client/Server 등을 쉽게 개발할 수 있는 Spring AI를 활용하여 Agentic AI를 구현하고 연동함

유연한 AI 개발 및 연동 환경 구성

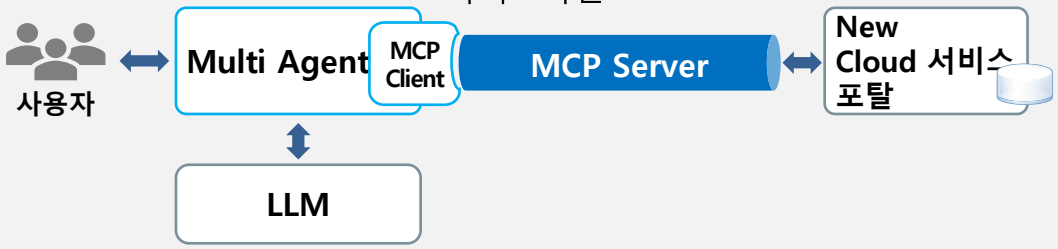
출처 *사내 AI 에코시스템 아키텍처(Flowise·Spring AI·MCP)*

AI 서비스 개발 방안

MCP가 실행을 담당하는 MCP 아키텍처의 Agentic AI 서비스를 기존 MSA 구조와 유연하게 연결되도록 개선 개발함으로써 향후 AI 플랫폼으로 진화하는 초석을 마련합니다.

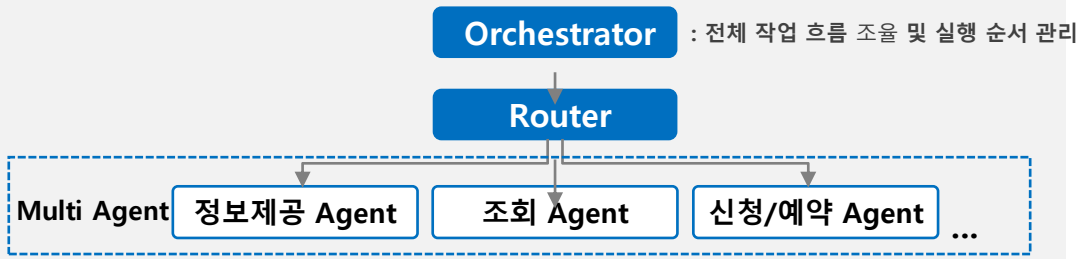
MCP 구성

"서비스별 단절된 데이터를 AI 모델과 연결하는 표준화된 파이프라인"



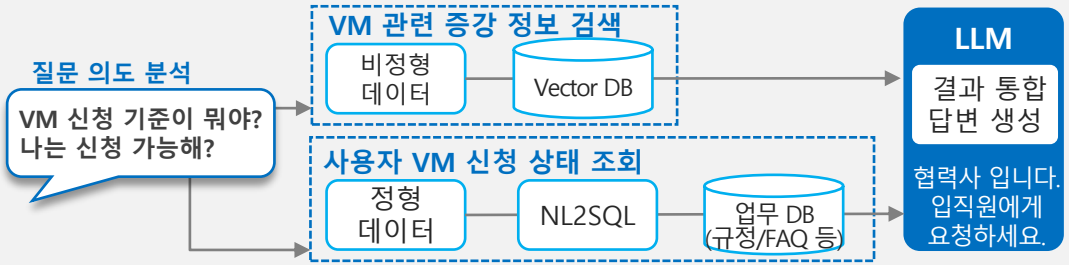
Agentic AI Workflow

"정의된 절차 기반 작동으로 구현의 안정성과 효율성 제공"



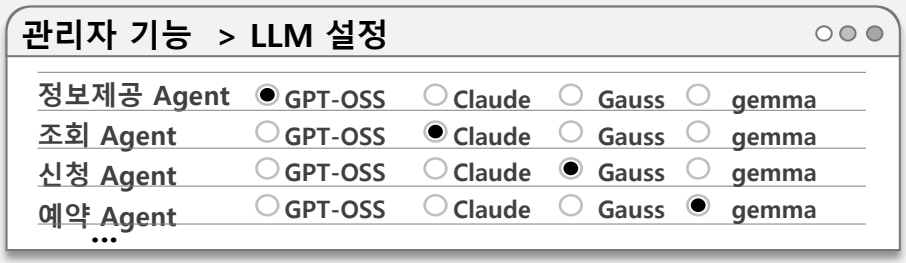
Hybrid RAG 기반 AI 지식 검색 구조

"비정형 데이터 RAG + NL2SQL 정형 데이터조회 = 답변 정확도 향상"



다양한 LLM 활용

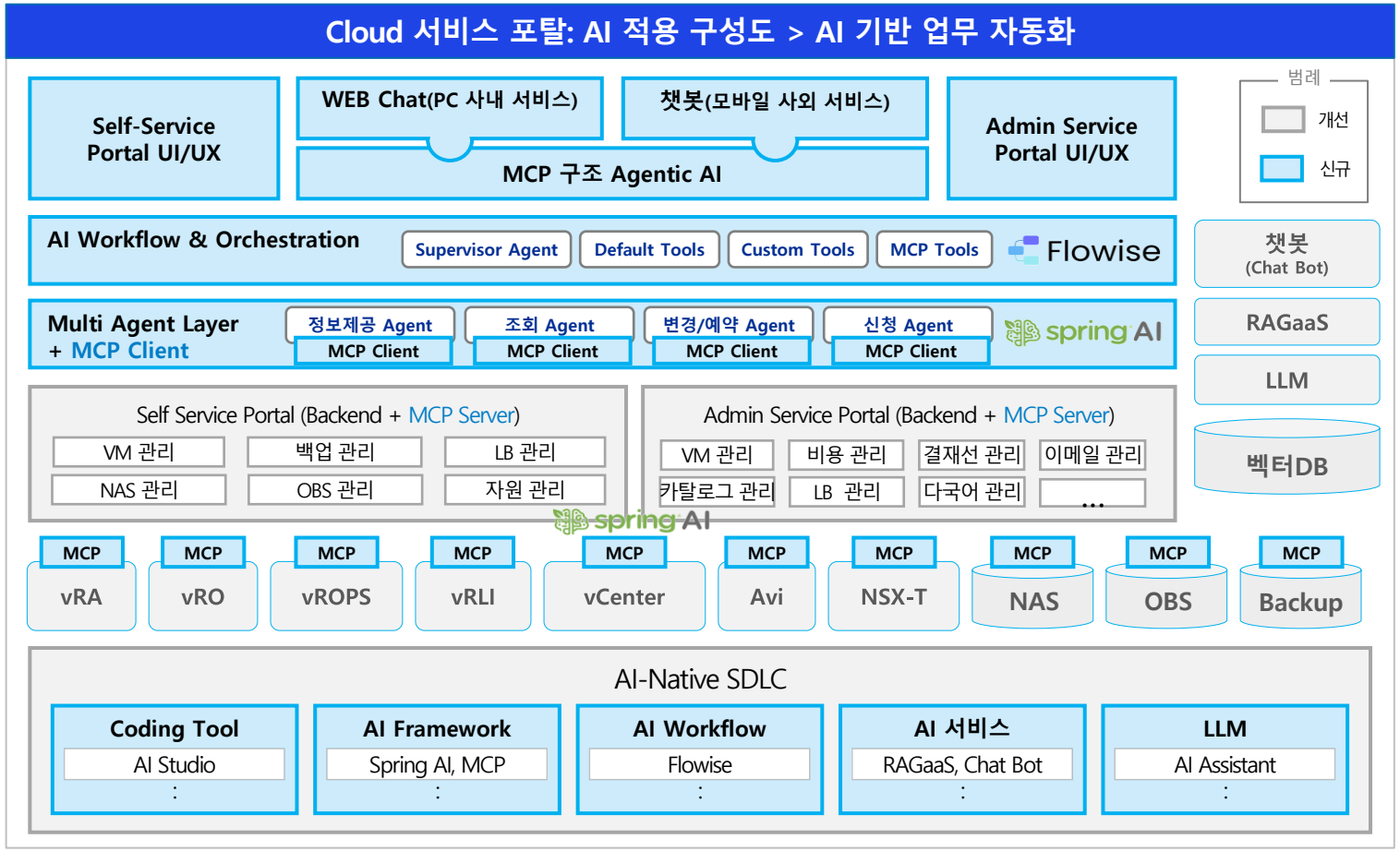
"각 Agent에 최적화된 LLM을 사용할 수 있도록 관리자 기능 제공"



출처 *사내 Agentic AI 서비스 아키텍처(MCP·Multi-Agent)*

AI 에코시스템 적용 사례

Cloud 서비스 포탈은 기존의 MSA 기반 서비스에서 제공되었던 품질 수준을 안정적으로 유지하면서 AI 활용 등 최적의 기술 스택과 아키텍처로 사용자 중심 서비스 및 내/외부 인터페이스에 대한 개선 사항을 누락 없이 효과적으로 반영하여 구현합니다.



AI 적용 방향

- AI를 통한 사용자 UI/UX 편의성과 서비스 접근성 증대
- AI Web Chat 통한 대화형 조회/추천/신청 서비스 제공
- Multi Agentic AI를 위한 Orchestration Layer 적용
- 다양한 업무 기능을 제공하는 Multi Agent 설계/개발
- Cloud 서비스 포탈에 대한 개선 및 MCP 전환
- REST 기반 인프라 서비스에 대한 MCP 전환

문제점 및 개선 필요성

사용자 측면	• 검색과 추천 부재, 낮은 접근성, 불편한 UX/UI
기술적 측면	• 레거시 기술, 유지보수 어려움, 확장성 부족
운영 측면	• 수작업 처리, 반복 업무, 비효율적 관리
데이터 측면	• 디지털 전환 및 활용 미흡, 인사이트 부재

출처 사내 Cloud 서비스 포탈 AI 에코시스템 적용 사례

다양한 모델 연동 실습 개요

Claude Code로 여러 제공자의 모델을 연동해 같은 프롬프트로 비교·선택합니다.

1 Anthropic Claude

- Messages API
- claude-sonnet/opus
- 코드·추론 강점

2 OpenAI / Azure

- Chat Completions
- gpt-4o / mini
- 범용·생태계

3 Google Gemini

- generateContent
- gemini
- 멀티모달·검색

4 로컬 OSS

- Ollama
- Llama·Qwen·gpt-oss
- 무료·온프레미스

출처 Anthropic / OpenAI(Azure) / Google / Ollama 공식 API 문서

03

1주차 | SECTION

AI 개발 환경 구성과 활용

Python · 가상환경 · API 키 · AI 코딩 도구 — 첫 개발 환경을 만든다.

주요 AI 코딩 도구

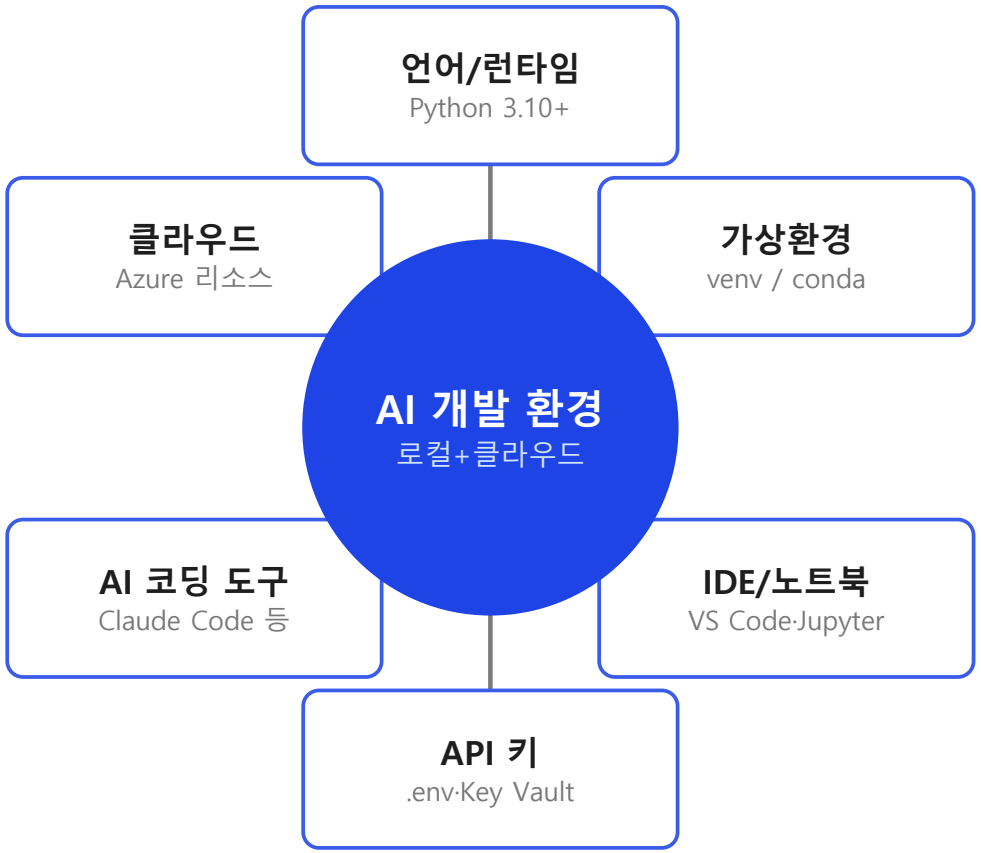
터미널·IDE·클라우드 등 형태별로 AI 코딩 도구를 비교해 선택합니다.

도구	유형	특징
Claude Code	터미널 Agentic CLI	대규모 코드베이스·리팩터링 강점
GitHub Copilot	IDE 통합	인라인 자동완성·범용
Cursor	AI 코드 편집기	IDE 내 멀티파일 편집
OpenAI Codex	클라우드 에이전트	비동기·병렬 작업
Windsurf / Amazon Q	에이전트 편집기	Cascade·클라우드 통합

출처 AI 코딩 솔루션 비교·확장분석(9종) 사내자료 · 각 도구 공식 문서(2025~2026) 기준

AI 개발 환경 구성요소

개발 환경은 언어·격리·키 관리에 AI 코딩 도구가 더해집니다.



핵심 설명

1. **언어/런타임**
   Python 3.10+ 등 실행 환경
2. **가상환경**
   venv/conda로 의존성 격리
3. **IDE/노트북**
   VS Code·Jupyter로 개발·실험
4. **API 키 관리**
   .env·Key Vault로 안전하게 보관
5. **AI 코딩 도구**
   Claude Code 등으로 생산성 향상

출처 *AI 코딩 솔루션 비교분석(사내) · Azure AI 개발 가이드*

전통 개발 → AI 보조 개발

AI 코딩 도구는 개발 생산성과 일관성을 높입니다.

전통 개발

- 사람이 코드를 직접 작성
- 문서·테스트는 사후
- 반복 작업 수작업
- 개인 역량에 의존

AI 코딩 도구 도입

AI 보조 개발

- AI가 코드 초안·리팩터링
- 코드+테스트 동시 생성
- 반복 작업 자동화
- 도구·컨텍스트로 표준화

출처 *AI 코딩 솔루션 비교분석(사내)*

개발 환경 구성 절차



출처 *Azure AI 개발 가이드 · AI 코딩 솔루션 비교분석(사내)*

안전한 키 관리 원칙

보안 사고의 가장 흔한 원인은 키 유출입니다. 처음부터 습관을 들입니다.

- API 키는 코드에 하드코딩하지 않는다 — .env / 환경변수 사용
  - .gitignore에 .env를 추가해 유출 방지
- 키는 최소 권한·주기적 회전, 비용 한도(쿼터) 설정
- 운영 환경은 Key Vault 등 비밀 관리 서비스 활용

Claude Code 실습 개요 (33·35p)

33·35페이지의 환경 구성과 첫 API 호출을 Claude Code 프롬프트·명령으로 실습합니다.

1 환경 구성 (33p)

- venv·패키지·.env
- 프롬프트로 자동 구성

2 첫 API 호출 (35p)

- 코드 생성·실행
- Claude Code가 처리

3 보안·견고성

- .gitignore·재시도
- 프롬프트로 개선

출처 *Anthropic Claude Code Docs*

04

1주차 | SECTION

에이전틱 코딩 입문

AI가 코드를 작성하는 시대 — 도구와 패러다임을 미리 본다.

Vibe Coding vs Agentic Coding

AI 코딩 도구는 "생성"을 넘어 "검증된 산출물"로 무게중심이 옮겨갑니다.

Vibe Coding

- "일단 만들어줘" 즉흥 생성
- 한 번에 생성·수정
- 의도에서 drift
- happy path 위주

패러다임 전환

Agentic Coding

- 목표·검증 기준 설정
- 에이전트가 탐색·수정
- 자동 검증·재시도
- 사람은 감독자

출처 *에이전틱 코딩 & SDD 개요(사내)*

에이전틱 코딩 도구 지형

여러 AI 코딩 도구가 "명세(spec) 정본" 구조로 수렴하고 있습니다.

1 GitHub Spec Kit

- /specify→/plan→/tasks
- 오픈소스 툴킷
- 명세를 명령으로

2 Amazon Kiro

- Requirements→Design→Tasks
- 클라우드 IDE 내장
- AWS 연계

3 Tessl

- spec = 최종 산출물
- 코드가 아닌 spec 유지
- 명세 중심

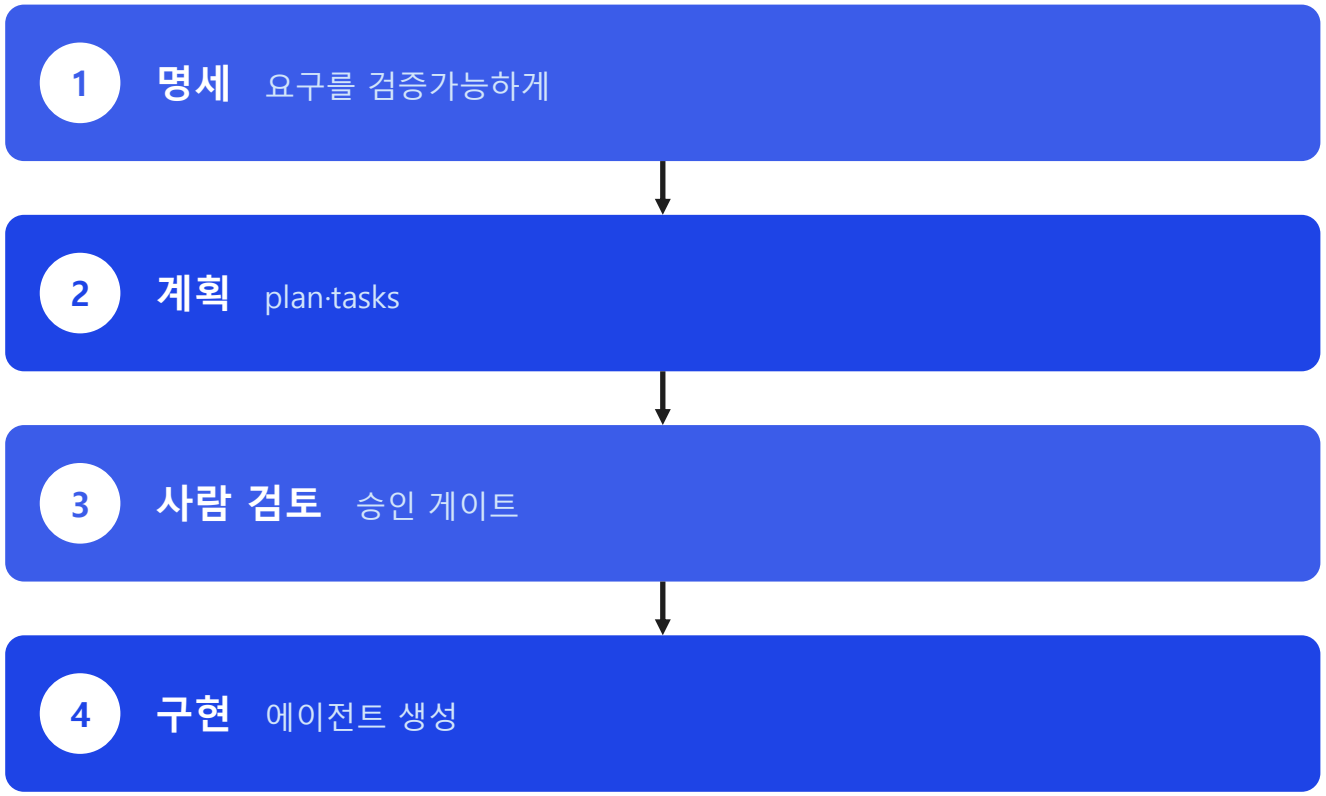
4 Claude Code

- 터미널 Agentic CLI
- 스킬·서브에이전트
- 로컬 코드베이스

출처 SDD 도구 지형(사내) · 각 도구 공식 문서

AI 코딩 도구 공통 골격

이름은 달라도 "명세 먼저 → 사람 검토 → 구현" 골격은 동일합니다.



핵심 설명

1. **명세**
   흐릿한 요구를 검증 가능한 spec으로 변환
2. **계획**
   plan·tasks로 실행 단위 분해
3. **사람 검토**
   승인 게이트 통과 전 구현 0건
4. **구현**
   에이전트가 명세대로 생성
5. **공통점**
   Spec Kit·Kiro·Claude Code 모두 동일 흐름

출처 SDD 도구 공통 골격(사내)

에이전틱 코딩이 바꾸는 3가지

에이전틱 코딩은 속도·일관성·추적성을 동시에 끌어올립니다.

1 무게중심

- 생성 속도 → 검증·증빙
- "돌아간다"가 아닌 "맞다"

2 drift 방지

- 명세가 의도를 고정
- 긴 작업에서 표류 차단

3 추적성

- 명세·검증 기록 보존
- PR 단위 책임 경계

출처 *에이전틱 코딩 & SDD 개요(사내)*

1주차 핵심 정리

- ① 클라우드(IaaS·PaaS·SaaS)와 AI가 결합 — 전통 IT에서 데이터·모델 중심의 X-AI Native로 전환
- ② AI 생태계는 인프라–모델–프레임워크–앱 4계층 — Claude·GPT·Gemini·로컬 OSS 등 다양한 모델을 연동·비교
- ③ 개발 환경은 가상환경·안전한 키 관리에서 시작 — AI 코딩 도구로 첫 API 호출까지 자동화
- ④ 에이전틱 코딩(명세 주도)·Claude Code·MCP로 개발을 자동화·표준화한다

▶ 다음 차시 2주차 — AI 기반 인프라·애플리케이션·데이터 활용 기초